

1. Введение

Хочется сначала показать логику развития этой теории: откуда возникла концепция жадноидов, какую проблему она решает и почему эти структуры имеют такое важное значение в дискретной математике.

Развитие вычислительной техники ясно показало: чтобы алгоритм работал эффективно, он должен в полной мере эксплуатировать внутреннюю структуру исследуемого математического объекта. Если в непрерывной математике залогом успешной оптимизации часто служит выпуклость функции, то в дискретной математике связь между структурой и алгоритмом проявляется еще более прямолинейно. Один из главных вопросов комбинаторной оптимизации звучит так: в каких случаях простое локальное правило гарантированно приводит к глобальному оптимуму?

Исторически первый ответ на этот вопрос был найден еще до появления современных компьютеров, при решении сугубо прикладной задачи. В 1926 году Отакар Боровка искал оптимальный способ электрификации Моравии. Он смоделировал это как задачу о минимальном остовном дереве и решил ее, шаг за шагом выбирая локально лучшие варианты. Принцип, который он математически обосновал, сегодня известен нам как жадный алгоритм.

Позже, в 1935 году, Хасслер Уитни ввел понятие матроида — абстрактной структуры, обобщающей свойства линейной независимости векторов и циклов в графах. Вскоре выяснилось, что именно матроиды являются математической базой, на которой жадные алгоритмы работают безупречно. Эта связь оказалась настолько глубокой, что само определение матроида можно дать чисто алгоритмически.

Поскольку теория матроидов оказалась невероятно успешной и породила множество мощных минимаксных теорем, математики начали искать ее обобщения. Если матроиды — это комбинаторная абстракция линейной независимости, то как быть с другими фундаментальными свойствами, например, с выпуклостью? Эта мысль привела к созданию антиматроидов. По своим свойствам они во многом противоположны матроидам. Антиматроиды естественно описывают, например, процессы составления расписаний или процедуру сборки: мы можем выполнить задачу или добавить деталь только тогда, когда выполнено хотя бы одно из предшествующих условий.

И здесь мы подходим к центральному объекту этой теории. Жадноиды (greedoids) — это концепция, которая объединяет в себе свойства и матроиды, и антиматроиды. В матроидах любое подмножество независимого множества обязано быть независимым. Жадноиды отказываются от этого требования. Для них ключевым становится свойство достижимости: любое допустимое множество можно вырастить из пустого, пошагово добавляя элементы по одному, и на каждом шаге оставаться в рамках допустимости. Поскольку порядок добавления элементов здесь имеет значение, жадноиды часто удобно рассматривать не как системы множеств, а как наборы упорядоченных последовательностей. Стоит отметить один важный нюанс: на жадноидах жадный алгоритм гарантированно находит оптимум для класса так называемых «узких» (bottleneck) целевых функций. Однако оптимизация обычной линейной функции на жадноидах в общем случае является NP -трудной задачей.

2. Базовые понятия

Система множеств над конечным базовым множеством E представляет собой пару $(E, \mathcal{F})$, где $\mathcal{F} \subseteq 2^E$. Множества, принадлежащие семейству $\mathcal{F}$, называются **допустимыми** (feasible), а остальные — недопустимыми.

Так как порядок добавления элементов часто имеет значение, множество E рассматривается как алфавит, его элементы — как буквы, а их последовательности — как слова. **Языком** $\mathcal{L}$ называется произвольный набор слов над алфавитом E . Язык называется **простым**, если ни одна буква не повторяется внутри одного слова, и **нормальным**, если он не содержит букв, не входящих ни в одно слово языка.

Ключевым для формализации процессов пошагового построения является понятие **наследственного языка** (hereditary language). Это язык, который удовлетворяет двум аксиомам: во-первых, он содержит пустое слово (обозначается $\emptyset$); во-вторых, он замкнут относительно взятия префиксов.

Каждому простому наследственному языку можно сопоставить систему множеств, элементами которой являются множества букв, образующих слова языка. Полученная система множеств называется **достижимой**. Она обладает следующими свойствами: содержит пустое множество, и для любого непустого допустимого множества $X \in \mathcal{F}$ существует элемент $x \in X$ такой, что $X \setminus \{x\}$ также является допустимым. Это описывает идею «выращивания» решений из пустого множества.

Наследственные языки обладают более богатой структурой, чем достижимые системы множеств: разные языки могут порождать одну и ту же систему. Однозначное соответствие между ними достигается только в том случае, если язык удовлетворяет дополнительному условию: если два слова языка образованы одним и тем же множеством букв с добавлением одного нового элемента, то слово, полученное присоединением этого нового элемента ко второму слову, также должно принадлежать языку.

Как и в теории матроидов, для анализа алгоритмов нам необходимо понимать, как выглядят финальные (максимальные) решения. Для заданной системы множеств вводится понятие **базы** — это максимальное по включению допустимое подмножество, которое можно собрать внутри некоторого заданного множества $A \subseteq E$. Если перевести это на язык процессов, база — это состояние, когда алгоритм добавил всё, что мог, и больше ни один элемент добавить нельзя без

нарушения правил. Аналогичным образом определяются **базовые слова** для языков — это максимально длинные корректные последовательности шагов.

Для изучения взаимосвязей между различными множествами (например, между путями и разрезами в графе) полезным инструментом является **клаттер**. Клаттер — это семейство множеств, в котором ни одно множество не содержит другое. Хороший пример клаттера — семейство всех минимальных остовных деревьев графа: ни одно дерево не является подмножеством другого.

Для любого клаттера определяется его **блокер** — это семейство минимальных множеств, которые имеют непустое пересечение с каждым элементом исходного клаттера. Иными словами, элемент блокера гарантированно «задевает» каждое множество из клаттера. Важным и красивым свойством этой операции является инволютивность: если мы построим блокер для блокера, мы всегда в точности получим исходный клаттер.

Помимо систем множеств, для понимания жадноидов (и особенно антиматроидов, описывающих расписания) требуется аппарат теории частично упорядоченных множеств (посетов) и решеток.

В частично упорядоченном множестве $(E, \leq)$ отношение $\leq$ удобно воспринимать как зависимость (например, задачу x нужно выполнить до задачи y). В таком множестве выделяются два важных типа подмножеств:

- **Идеалы** — это подмножества, «замкнутые вниз». Если идеал содержит элемент, он обязан содержать и всех его предшественников. В терминах расписаний, идеал — это корректный набор уже выполненных задач.
- **Фильтры** — это подмножества, «замкнутые вверх». Они содержат элемент вместе со всеми его последователями (это задачи, которые еще предстоит выполнить).

Элементы посета могут быть связаны по-разному. Подмножество попарно сравнимых элементов образует **цепь**. Подмножество попарно несравнимых элементов образует **антицепь**. Связь между ними описывается **теоремой Дилуорса**: минимальное число цепей, необходимых для покрытия всего множества, в точности равно максимальному размеру антицепи.

Частичный порядок становится **решеткой** (lattice), если для любых двух его элементов всегда существует:

- Точная нижняя грань (**meet**, $\wedge$) — наибольший общий предшественник двух элементов.
- Точная верхняя грань (**join**, $\vee$) — наименьший общий последователь двух элементов.

В конечной решетке всегда присутствуют единственный минимальный элемент (0, соответствующий стартовому пустому состоянию) и максимальный элемент (1, соответствующий полностью завершённому процессу).

3. Матроиды

Сначала немного концепции:

Математики заметили, что в разных областях есть системы, которые ведут себя одинаково.

В векторах: Если у вас есть набор векторов, вы хотите выбрать из них базис — максимальное количество векторов, которые не «дублируют» друг друга (ЛНЗ).

В графах: Если у вас есть граф, вы хотите выбрать максимальный набор ребер, который не образует циклов (остовное дерево). Лишнее ребро, замыкающее цикл, — это «дубликат» пути.

Смысл матроида в том, что он абстрагирует это понятие «независимости». Если мы назвали какое-то множество «независимым», это значит, что в нем нет ничего лишнего, никакой внутренней избыточности.

Матроиды аксиоматизируют свойства линейной зависимости векторов и циклов в графах. Система множеств $(E, \mathcal{M})$ называется матроидом, если:

- $\emptyset \in \mathcal{M}$;
- **Наследственность**: если $X \in \mathcal{M}$ и $Y \subseteq X$, то $Y \in \mathcal{M}$ (система, удовлетворяющая этих двум аксиомам, называется **независимой**);
- **Аксиома дополнения**: если $X, Y \in \mathcal{M}$ и $|X| = |Y| + 1$, то существует $x \in X \setminus Y$ такой, что $Y \cup \{x\} \in \mathcal{M}$.

Последняя аксиома дополнения гарантирует, что все базы (максимальные допустимые множества) имеют одинаковую мощность (если где-то существует множество большего размера, вы всегда можете «подтянуться» до его размера, забирая у него элементы). Эта мощность называется **рангом** $r(A)$ подмножества A .

Важнейшей характеристикой матроида является **оператор замыкания** $\sigma(X)$, который добавляет к множеству X все элементы, не увеличивающие его ранг. В матроидах этот оператор обладает **свойством обмена Стейница-Маклейна**: если $y \notin \sigma(X)$, но $y \in \sigma(X \cup \{x\})$, то $x \in \sigma(X \cup \{y\})$.

С точки зрения оптимизации, матроиды — это в точности те независимые системы, на которых жадный алгоритм находит максимум **линейной** целевой функции $c(S) = \sum_{e \in S} c(e)$ для любых положительных весов.

Теперь немного интуиции:

Любая задача оптимизации — это поиск лучшего среди допустимых (что значит допустимые мы определяли ранее). Если мы ищем кратчайший путь, то «допустимые множества» — это наборы ребер, которые образуют путь от А до Б. Набор ребер, который ведет в тупик — недопустим.

В матроиде легален фокус: один тяжелый элемент может вытеснить не более одного другого элемента из оптимального набора. Это обеспечивается аксиомой дополнения. Если жадный алгоритм выбрал какой-то тяжелый элемент g , а в оптимальном решении его нет, то благодаря аксиоме дополнения мы всегда можем провести замену.

1. Пусть жадный алгоритм выбрал набор $G = \{g_1, g_2, \dots, g_k\}$.
2. Пусть есть какой-то другой оптимальный набор $O = \{o_1, o_2, \dots, o_k\}$.
3. Если самый первый (самый тяжелый) элемент жадника g_1 не входит в O , то по аксиоме дополнения можно дополнить множество $\{g_1\}$ еще $k - 1$ элементами до размера базы. Тогда будет существовать единственный элемент o^* , которые не вошел в новый набор.
4. Получится новый набор O' , который:
Тоже является независимым (это гарантирует матроид).
Весит не меньше, чем O (потому что g_1 — самый тяжелый, он весит как минимум столько же, сколько o^*). Однако весить новый набор больше, чем оптимальный не может. Значит набор O' также является оптимальным, то есть существует решение, которое включает в себя g_1 (заменили o^* на g_1).
5. Повторим предыдущие шаги - получим оптимальное решение.

Теперь про единственность:

Если нарушется аксиома дополнения, например существует оптимум X из трех элементов с весами по 9, а также множество Y из двух элементов с весами по 10, но при это из Y не дойти в X , то жадник выберет сначала два элемента из Y , но поскольку размера базы не достичь, то жадник остановится и оптимальное решение достигнуто не будет.

4. Антиматроиды

Если матроиды являются комбинаторным обобщением линейной независимости, то **антиматроиды** представляют собой абстракцию понятия **выпуклости**. В то время как матроиды симметричны и позволяют выбирать элементы в произвольном порядке, антиматроиды описывают структуры с жесткой очередностью действий, где одни элементы открывают доступ к другим.

Система множеств $(E, \mathcal{F})$ называется антиматроидом, если она удовлетворяет двум ключевым условиям:

1. **Достижимость:** Пустое множество допустимо ($\emptyset \in \mathcal{F}$), и для любого непустого допустимого множества $X \in \mathcal{F}$ существует такой элемент $x \in X$, что $X \setminus \{x\} \in \mathcal{F}$.

Это условие связывает антиматроиды с теорией языков: любое допустимое множество можно представить как «базовое слово», которое можно «собрать» из пустого множества, добавляя элементы по одному и всегда оставаясь в рамках допустимых состояний. В отличие от матроидов, антиматроиды не обязаны быть наследственными, но они обязаны быть «собираемыми».

2. **Замкнутость относительно объединения.**

4.1. Shelling-процесс и выпуклость

Наилучшей визуализацией антиматроида является процесс **shelling** (снятие оболочек). Представим конечное множество точек на плоскости. Допустимым действием назовем удаление точки, если она является вершиной выпуклой оболочки текущего множества.

- В начале мы можем удалить только «крайние» точки.
- Удаление одной точки может сделать «крайней» ту точку, которая раньше находилась внутри.
- Множество всех удаленных к данному моменту точек будет допустимым множеством антиматроида.

Именно поэтому антиматроиды называют абстрактной выпуклостью: элементы, которые можно добавить (или удалить) прямо сейчас, аналогичны экстремальным точкам выпуклого тела.

4.2. Анти-обмен

Свое название антиматроиды получили из-за **свойства анти-обмена**, которое математически противоположно свойству Стейница в матроидах. Если через $\tau(X)$ обозначить оператор замыкания (в контексте антиматроидов это аналогично выпуклой оболочке), то выполняется следующее:

Если $y, z \notin \tau(X)$, но $z \in \tau(X \cup \{y\})$, то $y \notin \tau(X \cup \{z\})$.

Это формализует идею односторонней зависимости. Если выполнение задачи y открывает возможность выполнения задачи z , то в этой структуре невозможно обратное — чтобы z открывала доступ к y . Это свойство антисимметрии определяет направленность процессов в антиматроидах.

4.3. Оптимизация Bottleneck-функций

В матроидах жадный алгоритм оптимизирует сумму весов (линейную функцию). В антиматроидах ситуация иная: жадный алгоритм эффективно находит оптимум для так называемых **bottleneck-функций**.

- Вес множества определяется как $W(X) = \min_{x \in X} c(x)$ или $W(X) = \max_{x \in X} c(x)$.
- Жадная стратегия: на каждом шаге среди всех доступных элементов (тех, что сохраняют достижимость) выбрать элемент с наилучшим весом.

Поскольку возможности в антиматроиде только расширяются (замкнутость по объединению), локально лучший выбор никогда не заблокирует доступ к глобально оптимальному элементу (если у нас есть наше текущее решение B и оптимальное A , то взяв на очередном шаге минимум из доступных элементов мы всегда сможем взять и элемент из A , поскольку $A \cup B \in F$).

4.4. Поисковые антиматроиды

Одним из наиболее интуитивных примеров антиматроида является процесс поиска в графе. Пусть задан связный граф $G = (V, E)$ с выделенным корнем r . Определим систему множеств $(V \setminus \{r\}, \mathcal{F})$, где допустимыми множествами $\mathcal{F}$ являются наборы вершин, которые вместе с корнем образуют связный подграф.

Связь с алгоритмами поиска: Алгоритмы обхода графа, как DFS или BFS, по сути являются различными стратегиями построения *базового слова* в этом антиматроиде:

- **Условие доступности:** Вершина v становится доступной для добавления в текущее множество X тогда и только тогда, когда у неё есть хотя бы один сосед в $X \cup \{r\}$. Это в точности соответствует DFS: нельзя дойти до вершины, не посетив её предка.
- **Замкнутость по объединению:** Если наборы вершин A и B достижимы из корня по отдельности, то их объединение $A \cup B$ также образует связную структуру: посещение одной ветки никогда не блокирует возможность посещения другой.

Докажем корректность DFS. Под корректностью DFS мы понимаем его способность посетить все вершины в компоненте связности стартового узла r . Рассмотрим поисковый антиматроид $(E, \mathcal{F})$, где $E = V \setminus \{r\}$, а $\mathcal{F}$ — семейство подмножеств вершин, которые вместе с r образуют связный подграф.

Лемма 1. Пусть $G = (V, E)$ — связный граф, $r \in V$ — корень. Система $(V \setminus \{r\}, \mathcal{F})$, где $\mathcal{F} = \{X \subseteq V \setminus \{r\} : G[X \cup \{r\}] \text{ связен}\}$, является антиматроидом.

Доказательство. Проверим выполнение аксиом антиматроида:

1. Достижимость:

- Для пустого множества $X = \emptyset$ индуцированный подграф состоит только из вершины r , которая по определению связна. Значит, $\emptyset \in \mathcal{F}$.
- Пусть $X \in \mathcal{F}$ и $X \neq \emptyset$. Это значит, что подграф $G' = G[X \cup \{r\}]$ связен. В любом связном графе с числом вершин $n \geq 2$ существует хотя бы одно остовное дерево, а в этом дереве есть хотя бы лист. Пусть x — один из этих листов. Удаление этого листа из связного графа оставляет его связным. Следовательно, подграф $G[(X \setminus \{x\}) \cup \{r\}]$ остается связным, и $X \setminus \{x\} \in \mathcal{F}$.

2. Замкнутость по объединению:

Пусть $A \in \mathcal{F}$ и $B \in \mathcal{F}$. Это означает, что:

- В подграфе $G[A \cup \{r\}]$ каждая вершина $a \in A$ имеет путь до корня r .
- В подграфе $G[B \cup \{r\}]$ каждая вершина $b \in B$ имеет путь до корня r .

Рассмотрим объединение $G[(A \cup B) \cup \{r\}]$. Любая вершина v из этого объединения принадлежит либо A , либо B (либо корню). В обоих случаях у неё сохраняется путь до r , проходящий только через вершины из $A \cup B \cup \{r\}$. Раз каждая вершина имеет путь до корня, весь индуцированный подграф $G[(A \cup B) \cup \{r\}]$ связен. Значит, $A \cup B \in \mathcal{F}$. $\square$

Значит система связных подграфов, содержащих корень, удовлетворяет аксиомам достижимости и замкнутости по объединению, а значит, является антиматроидом.

Теорема 1. DFS, стартующий из вершины r в связном графе G , завершается тогда и только тогда, когда множество посещенных вершин X совпадает с $V \setminus \{r\}$.

Доказательство. Заглушка

1. По определению антиматроида, семейство $\mathcal{F}$ замкнуто относительно операции объединения: если $A, B \in \mathcal{F}$, то $A \cup B \in \mathcal{F}$. Следовательно, в антиматроиде существует единственное максимальное по включению допустимое множество (база) $X_{\max} = \bigcup_{X \in \mathcal{F}} X$. Для связного графа это множество содержит все вершины графа, кроме корня ($X_{\max} = V \setminus \{r\}$). Любой процесс расширения допустимого множества в антиматроиде приводит к достижению X .

2. Согласно аксиоме дополнения для антиматроидов, для любых двух допустимых множеств $X, Y \in \mathcal{F}$, если $X \not\subseteq Y$, то существует такой элемент $x \in X \setminus Y$, что $Y \cup \{x\} \in \mathcal{F}$.

Пусть Y — текущее множество вершин, посещенных алгоритмом DFS. Если $Y \neq X_{\max}$, то, согласно аксиоме, мы всегда можем найти вершину x из полного решения $X_{\max}$, которую можно добавить к нашему текущему набору Y , не нарушая связности. Это означает, что пока не посещены все достижимые вершины, у алгоритма всегда существует хотя бы один валидный вариант продолжения (сосед уже посещенной вершины).

3. DFS на каждом шаге выбирает вершину v , которая смежна с уже посещенным множеством Y . Это гарантирует, что на каждом шаге i множество Y_i остается связным с корнем, то есть $Y_i \in \mathcal{F}$.

Поскольку:

- на каждом шаге размер Y строго увеличивается;
- свойство достижимости гарантирует наличие продолжения до момента $Y = X_{\max}$;

алгоритм обязан завершиться в состоянии $Y = X_{\max}$. $\square$

5. Жадноиды

Жадноид — это наиболее общая структура, рассматриваемая в данной теории. Он объединяет в себе свойство **достижимости** (из антиматроидов) и **аксиому дополнения** (из матроидов), но при этом отказывается от требования наследственности.

Система множеств $(E, \mathcal{F})$ называется жадноидом, если она удовлетворяет следующим условиям:

1. **Достижимость:** $\emptyset \in \mathcal{F}$, и для любого непустого $X \in \mathcal{F}$ существует $x \in X$ такой, что $X \setminus \{x\} \in \mathcal{F}$.
2. **Аксиома дополнения:** Если $X, Y \in \mathcal{F}$ и $|X| > |Y|$, то существует такой элемент $x \in X \setminus Y$, что $Y \cup \{x\} \in \mathcal{F}$.

Таким образом, жадноид — это матроид без наследственности. В нем можно строить решение пошагово (достижимость), и мы гарантированно не застрянем в локальном тупике, если где-то существует решение большего размера (аксиома дополнения).

5.1. Почему это работает?

Жадноиды решают следующие проблемы:

- От матроидов они наследуют возможность использовать жадный алгоритм для достижения максимального размера базы. Поскольку $|X| > |Y|$ всегда позволяет расширить Y , все базы жадноида имеют одинаковый ранг.
- От антиматроидов они наследуют структуру зависимости от порядка.

У нас пропадает наследственность: в дереве, растущем из корня, нельзя выкинуть ребро из середины — дерево распадется и перестанет быть допустимым. Но можно удалять листья. Это и есть достижимость без полной наследственности.

5.2. Оптимизация на жадноидах

Линейная целевая функция имеет вид $W(X) = \sum_{e \in X} w(e)$. В матроидах жадность работает для этой функции благодаря свойству наследственности. В жадноидах наследственности нет, поэтому может возникнуть лажа в решении: выбор элемента с максимальным весом на текущем шаге может навсегда закрыть путь к подмножествам, которые в сумме дали бы значительно больший вес.

Оптимизация произвольной линейной функции на жадноидах является **NP-трудной** задачей. На жадноидах можно закодировать такие задачи, как поиск самого длинного пути в графе или задачу коммивояжера.

Жадноиды остаются хорошей структурой для оптимизации bottleneck-функций.

Почему это работает: Благодаря аксиоме дополнения, любое допустимое множество может быть достроено до базы. Поэтому выбор элемента с наилучшим весом из всех доступных на данном шаге гарантирует, что мы поддерживаем оптимальность нашего решения на максимально возможном уровне, не блокируя потенциально лучшие элементы в будущем.

Для того чтобы жадный алгоритм работал с суммой весов на жадноидах, весовая функция должна учитывать структуру достижимости. Такие функции называют **совместимыми с жадностью** (greedy-compatible).

Весовая функция w , определенная на словах (последовательностях) языка жадноида, совместима с жадностью, если выполняется условие локального превосходства:

$$\text{Если } w(\alpha x) \geq w(\alpha y), \text{ то для любого продолжения } \beta : w(\alpha x \beta) \geq w(\alpha y \beta)$$

Это означает, что локально лучший выбор x по сравнению с y не должен портить финальный результат больше, чем это сделал бы выбор y . Это условие выполняется, например, когда веса элементов сильно затухают в зависимости от их позиции в последовательности (каждый следующий шаг вносит значительно меньший вклад, чем предыдущий).

6. Операции над матроидами

6.1. Удаление и Ограничение

Пусть задан матроид $M = (E, \mathcal{M})$ и подмножество элементов $S \subseteq E$.

- **Restriction:** Обозначается $M|_S$. Новое семейство независимых множеств $\mathcal{M}_S$ состоит только из тех множеств исходного матроида, которые целиком лежат в S :

$$\mathcal{M}_S = \{X \in \mathcal{M} : X \subseteq S\}$$

- **Deletion:** Если мы удаляем множество T , это эквивалентно ограничению матроида на его дополнение: $M \setminus T = M|_{E \setminus T}$.

С точки зрения графов это просто выбрасывание ребер. С точки зрения алгоритмов это уменьшение «алфавита» поиска.

6.2. Стягивание (Contraction)

Базис множества X — это максимальное по включению независимое подмножество внутри X .

Матроид M/T на множестве элементов $E \setminus T$ определяется так: множество X независимо в стянутом матроиде, если оно может достроить какой-либо базис множества T до независимого множества в исходном матроиде.

- $M/T = \{X \subseteq E \setminus T : X \cup Y \in \mathcal{M} \text{ для некоторого базиса } Y \text{ множества } T\}$.

В графе это стягивание ребра. В линейной алгебре это проектирование пространства на подпространство, ортогональное вектору T . Мы как бы «вычитаем» информацию, которую несет T , из всей системы.

6.3. Объединение и Пересечение

- **Объединение** ($M_1 \vee M_2$): Множество независимо в объединении, если его можно разбить на два подмножества X_1 и X_2 , независимых в M_1 и M_2 соответственно. **Объединение матроидов всегда является матроидом**, и его ранг вычисляется по формуле Эдмондса:

$$r(X) = \min_{Y \subseteq X} \{r_1(Y) + r_2(Y) + |X \setminus Y|\}$$

- **Пересечение** ($M_1 \cap M_2$): Мы ищем множество, которое **одновременно** независимо в обеих системах. В общем случае **пересечение не является матроидом**, а значит, жадный алгоритм на нем не работает.

7. Субмодулярность и полиматроиды

7.1. Субмодулярные функции

Функция $f : 2^E \rightarrow \mathbb{R}$ называется **субмодулярной**, если для любых $X, Y \subseteq E$:

$$f(X \cup Y) + f(X \cap Y) \leq f(X) + f(Y)$$

С точки зрения алгоритмов это означает, что вклад одного и того же элемента e в «ценность» множества уменьшается по мере роста этого множества. Ранг любого матроида является субмодулярной функцией.

7.2. Полиматроиды

Полиматроид представляет собой обобщение концепции матроида, которое позволяет работать не только с дискретными наборами элементов, но и с вещественными векторами в пространстве $\mathbb{R}_+^{|E|}$.

Пусть E — конечное множество. Функция $f : 2^E \rightarrow \mathbb{R}$ называется **функцией ранга полиматроида**, если она удовлетворяет трем условиям:

1. **Нормировка:** $f(\emptyset) = 0$.

2. **Монотонность:** если $A \subseteq B \subseteq E$, то $f(A) \leq f(B)$.

3. **Субмодулярность:** для любых $X, Y \subseteq E$:

$$f(X \cup Y) + f(X \cap Y) \leq f(X) + f(Y)$$

Каждой такой функции f соответствует **многогранник полиматроида** P_f , который определяется как подмножество векторов $x = (x_e)_{e \in E} \in \mathbb{R}_+^{|E|}$, удовлетворяющих системе линейных неравенств:

$$P_f = \left\{ x \in \mathbb{R}^{|E|} : x \geq 0, \sum_{e \in S} x_e \leq f(S) \text{ для всех } S \subseteq E \right\}$$

Связь между матроидами и полиматроидами наиболее ярко проявляется при рассмотрении **характеристических векторов**. Это позволяет перевести задачу поиска оптимального множества на язык поиска точки в пространстве.

1. Характеристический вектор. Пусть E — базовое множество матроида, а $S \subseteq E$ — некоторое подмножество. Характеристическим вектором множества S называется вектор $\chi_S \in \{0, 1\}^{|E|}$, компоненты которого определяются следующим образом:

$$(\chi_S)_e = \begin{cases} 1, & \text{если } e \in S \\ 0, & \text{если } e \notin S \end{cases}$$

Таким образом, каждое подмножество матроида можно представить как точку в $|E|$ -мерном единичном кубе.

2. Целочисленность вершин. Рассмотрим многогранник полиматроида P_r , порожденный функцией ранга матроида $r(S)$:

$$P_r = \left\{ x \in \mathbb{R}^{|E|} : x \geq 0, \sum_{e \in S} x_e \leq r(S) \text{ для всех } S \subseteq E \right\}$$

Хотя этот многогранник определен в непрерывном пространстве $\mathbb{R}^{|E|}$ системой линейных неравенств, он обладает следующим свойством:

Все вершины (экстремальные точки) многогранника P_r имеют только целые координаты.

Более того, точка x является вершиной этого многогранника тогда и только тогда, когда $x = \chi_I$, где I — некоторое независимое множество матроида $\mathcal{M}$.

3. Почему это важно для оптимизации? Данный результат превращает дискретную задачу «выбора наилучшего набора элементов» в задачу линейного программирования:

- Вместо того чтобы перебирать все возможные независимые множества $I \in \mathcal{M}$, мы ищем максимум линейной функции $\sum c_e x_e$ на выпуклом множестве P_r .
- Так как максимум линейной функции на многограннике всегда достигается в одной из его вершин, а вершины здесь соответствуют независимым множествам, решение непрерывной задачи автоматически дает нам оптимальное дискретное решение.

Это геометрическое обоснование того, почему жадный алгоритм работает на матроидах. Вдоль ребер этого многогранника всегда можно перейти от одной вершины к другой, локально улучшая целевую функцию, и благодаря субмодулярной структуре граней мы никогда не застрянем в локальном максимуме, который не был бы глобальным.